

Technical Deep-Dive & Interview Guide

Task 1: Differentially Private NLP Text Obfuscation Pipeline

1. Executive Summary & Core Premise

Unstructured text datasets (such as IMDB movie reviews, clinical records, and customer communications) contain rich semantic information vital for machine learning models. However, raw text frequently exposes Personally Identifiable Information (PII) and sensitive contextual markers vulnerable to adversarial re-identification.

Traditional sanitization methods—such as Named Entity Recognition (NER) masking or regex redaction—fail to prevent linkage and membership inference attacks. This project implements a **Local Differential Privacy (LDP)** framework operating directly in dense vector space ($\mathbb{R}^d$). Continuous 50-dimensional GloVe word embeddings are perturbed using calibrated **Multivariate Laplacian Noise** derived from differential privacy metrics, guaranteeing provable mathematical privacy bounds while maintaining high downstream semantic utility.

2. Literature Review & Theoretical Foundations

2.1 Comparative Analysis of Text Privacy Paradigms

Privacy Model	Mechanism	Advantages	Critical Vulnerabilities
K-Anonymity & Masking	Rule-based entity removal / Regex substitution	Simple to implement, human-readable text	Vulnerable to background knowledge and high-dimensional linkage attacks.
Global DP (Centralized)	Noise added to query results or gradients (DP-SGD)	Strong mathematical guarantees	Requires an absolute trusted central server; raw data exposed at collection.
Local DP (Discrete)	Client-side discrete randomized response on tokens	Zero-trust required in central server	Catastrophic utility loss over large vocabularies (random output tokens).
Metric LDP (This Project)	Multivariate Laplacian noise injected into vector space $\mathbb{R}^d$	Preserves local semantic neighborhood and utility	Requires careful parameter calibration (epsilon vs. distance metric).

2.2 Theoretical Foundations of Local Differential Privacy

A randomized algorithm M provides ϵ -Differential Privacy if, for all neighboring inputs d, d' and all output subsets S :

$$\Pr[M(d) \in S] \leq \exp(\epsilon) \times \Pr[M(d') \in S]$$

Where ϵ (**Privacy Budget**) controls the privacy-utility trade-off. Lower ϵ values produce stronger privacy guarantees by increasing the variance of the injected noise. In continuous embedding space $\mathbb{R}^d$, noise vectors are drawn from a spherical distribution where direction is uniformly distributed on the unit sphere and magnitude follows a continuous Gamma distribution.

3. Deep-Dive Implementation & Code Architecture

3.1 Text Preprocessing & Cleaning Pipeline

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = re.sub(r'^a-zA-Z', ' ', str(text)).lower()
    tokens = word_tokenize(text)
    return [
        lemmatizer.lemmatize(w) for w in tokens
        if w not in stop_words and len(w) > 1
    ]
```

3.2 Multivariate Laplacian Noise Generation in R^d Space

```
import numpy as np

def generate_laplacian_noise_vector(dim=50, sensitivity=1.0, epsilon=25.0):
    # Step 1: Uniform random direction vector via standard Gaussian sampling
    gaussian_samples = np.random.normal(0, 1, dim)
    norm = np.linalg.norm(gaussian_samples)
    direction = gaussian_samples / norm if norm != 0 else gaussian_samples

    # Step 2: Radial magnitude from Gamma distribution
    scale = sensitivity / epsilon
    magnitude = np.random.gamma(shape=dim, scale=scale)

    return direction * magnitude
```

3.3 Obfuscation & Stochastic Top-k Semantic Replacement

```
from gensim.models import KeyedVectors

def replace_word(word, model, epsilon=25.0, sensitivity=1.0, top_k=5):
    if word not in model:
        return word # OOV Fallback

    original_vec = model[word]
    noise = generate_laplacian_noise_vector(dim=model.vector_size, sensitivity=sensitivity, epsilon=epsilon)
    noisy_vec = original_vec + noise

    try:
        similar_words = model.most_similar(positive=[noisy_vec], topn=top_k)
        candidates = [w for w, sim in similar_words]
        return np.random.choice(candidates) # Stochastic selection
    except Exception:
        return word
```

4. Key Technical Interview Q&A;

Q1: Why apply Local Differential Privacy in vector embedding space rather than on discrete tokens?

Answer: Discrete token perturbation on large vocabularies yields essentially random output tokens, causing catastrophic utility loss. By projecting text into continuous metric space ($\mathbb{R}^d$), distance corresponds to semantic similarity. Injecting noise into embeddings allows us to sample replacement tokens from the local semantic neighborhood, preserving grammar and utility.

Q2: How does the pipeline handle Out-Of-Vocabulary (OOV) tokens during processing?

Answer: OOV tokens are intercepted gracefully via lookup checks and try-except error handling. Unseen words are preserved without modification, preventing runtime exceptions and ensuring uninterrupted batch streaming on large datasets.

Q3: Why use stochastic selection among top-k nearest neighbors instead of always choosing k=1?

Answer: Deterministic nearest-neighbor replacement ($k=1$) creates a fixed mapping function. An adversary with access to the vocabulary embedding matrix could perform deterministic spatial inversion attacks. Random sampling among top-k candidates introduces non-deterministic entropy, breaking deterministic mapping attacks.

Q4: How would you optimize this architecture for high-throughput production workloads?

Answer: We can replace linear vector scans ($O(V \cdot d)$) with GPU-accelerated Approximate Nearest Neighbor (ANN) indexes like FAISS or Annoy ($O(\log V)$). Additionally, static GloVe embeddings can be upgraded to contextual Transformer representations (e.g., RoBERTa/BERT embeddings) processed in batched GPU memory chunks.